AES-128 Encryption: Milestone 1 Technical Guide

Comprehensive Architecture, Instruction Mapping, and Code Walkthrough for 8086 Assembly

Course: CSIS 402 - Computer Organization & System Programming	**Target CPU:** Intel 8086 (16-bit Architecture)
Milestone: 1 (Single Full Round AES-128)	**Environment:** MASM / TASM (DOSBox) or EMU8086
Deadline: Saturday 28/11/2026	**Key Used:** 16 bytes of 0FFh

1. Milestone 1 Objectives & Scope

Milestone 1 requires developing **four assembly macros** that together execute one complete encryption round of the Advanced Encryption Standard (AES-128) on a 128-bit block:

- **SubBytes()**: Non-linear byte substitution operating on the entire 128-bit state via the Rijndael S-Box.
- **ShiftRows()**: Cyclical left permutations applied to rows 1, 2, and 3 of the 4×4 state matrix.
- **MixColumns()**: Matrix-vector multiplication of each 4-byte column in the Galois Field GF(2■).
- **AddRoundKey()**: Bitwise XOR of the 128-bit state with the round key (fixed at 16 bytes of 0FFh).

2. Crucial Distinction: 128-Bit State vs. 256-Byte S-Box Table

A frequent point of confusion is why the assembly source contains a **256-byte S-Box table** when the project specification states that AES-128 operates on a **128-bit block**. The distinction is fundamental:

Concept	AES 128-Bit State (Data)	AES S-Box (Substitution Table)
Size	128 bits = **16 bytes**	256 bytes = **2048 bits**
Role	The working plaintext / ciphertext buffer being encrypted.	A static dictionary mapping every possible 8-bit value to a substitute byte.
Why this size?	AES-128 standardizes 128-bit plaintext blocks organized as a 4×4 matrix of bytes.	An 8-bit byte has 2■ = 256 distinct values (00h to FFh). To map any possible byte, 256 entries are required.

3. 128-Bit State Matrix & Memory Organization

In AES, the 16 bytes of data are treated as a two-dimensional 4×4 matrix. Crucially, the AES standard mandates a **column-major** arrangement. In 8086 linear memory, consecutive bytes represent consecutive rows within the same column:

Matrix Row	Column 0	Column 1	Column 2	Column 3

Row 0	state[0] (Offset +0)	state[4] (Offset +4)	state[8] (Offset +8)	state[12] (Offset +12)
Row 1	state[1] (Offset +1)	state[5] (Offset +5)	state[9] (Offset +9)	state[13] (Offset +13)
Row 2	state[2] (Offset +2)	state[6] (Offset +6)	state[10] (Offset +10)	state[14] (Offset +14)
Row 3	state[3] (Offset +3)	state[7] (Offset +7)	state[11] (Offset +11)	state[15] (Offset +15)

4. Detailed Technical Analysis of the Four Macros

4.1 SubBytes() Macro (Non-Linear Byte Substitution)

Concept: SubBytes provides non-linearity to confuse the relationship between the key and ciphertext. Each of the 16 state bytes is replaced by looking up its index in the 256-byte S-Box table.

8086 Assembly Implementation Technique: We utilize **Register Indirect Addressing** ([BX]) taught in Lecture 6. The base address of the S-Box table is placed in BX, the state byte value is zero-extended into AX, and BX is computed as BX = BX + AX. Then, MOV AL, [BX] retrieves the substitution byte in a single read without complex math.

```
SubBytes MACRO LOCAL sb_loop LEA SI, state ; SI = Source Index pointing to state MOV CX, 16 ; Loop counter for 16 bytes sb_loop: MOV AL, [SI] ; Load current byte MOV AH, 0 ; Zero-extend AL to AX LEA BX, sbox ; Base address of S-Box ADD BX, AX ; Effective address = sbox + byte_val MOV AL, [BX] ; Register Indirect lookup: sbox[byte] MOV [SI], AL ; Write back substituted byte INC SI ; Advance state pointer LOOP sb_loop ; CX = CX - 1; JNZ sb_loop ENDM
```

4.2 ShiftRows() Macro (Row Permutation)

Concept: ShiftRows introduces diffusion across columns by cyclically shifting each row to the left by an offset equal to its row index:

- **Row 0:** Shift 0 bytes (bytes at offsets 0, 4, 8, 12 remain untouched).
- **Row 1:** Shift left by 1 byte: state[1] ← state[5] ← state[9] ← state[13] ← state[1].
- **Row 2:** Shift left by 2 bytes: Equivalent to swapping pairs (2 ↔ 10) and (6 ↔ 14).
- **Row 3:** Shift left by 3 bytes: Mathematically identical to a circular right rotation by 1 byte: state[3] ← state[7] ← state[11] ← state[15] ← state[3].

Optimization: Rather than using nested loops and index calculations, our implementation uses direct register transfers (MOV AL, [SI+disp]). This eliminates branch prediction penalties and achieves minimal clock cycle latency on the 8086.

4.3 MixColumns() Macro & xtime Procedure (Galois Field Diffusion)

Concept: MixColumns provides vertical diffusion within each column by multiplying each column vector [s0, s1, s2, s3]■ with a fixed MDS matrix in the Galois Field GF(2■) modulo the Rijndael irreducible polynomial m(x) = x■ + x■ + x^3 + x + 1 (11Bh).

```
Matrix Transformation per Column: | r0 | | 02 03 01 01 | | s0 | r0 = (2*s0) XOR (3*s1) XOR s2 XOR s3 | r1 | = | 01 02 03 01 | * | s1 | ==> r1 = s0 XOR (2*s1) XOR (3*s2) XOR s3 | r2 | | 01 01 02 03 | | s2 | r2 = s0 XOR s1 XOR (2*s2) XOR (3*s3) | r3 | | 03 01 01 02 | | s3 | r3 = (3*s0) XOR s1 XOR s2 XOR (2*s3)
```

GF(2■) Arithmetic Properties:

- **Addition:** In GF(2■), addition is equivalent to bitwise XOR (there is no carry between bits).
- **Multiplication by 2 (xtime):** Implemented by shifting the byte left by 1 bit (SHL AL, 1). If the MSB (bit 7) was 1, it overflows into the Carry Flag (CF), meaning degree 8 was reached. We then reduce by XORing with 1Bh (the lower 8 bits of 11Bh).

- **Multiplication by 3:** Since 3 = 2 ⊕ 1 in GF(2■), 3*x = (2*x) XOR x = xtime(x) XOR x.

Temporary Column Storage (col_temp): Because all four equations for a column depend on the original values of s0, s1, s2, and s3, results cannot be written back to the state in-place until all four new bytes (r0, r1, r2, r3) are computed. We buffer them in col_temp.

4.4 AddRoundKey() Macro (Key Injection)

Concept: AddRoundKey combines the state with the round key using a bitwise XOR operation. For Milestone 1, the key is fixed at 16 bytes of 0FFh.

Mathematical Property: Bitwise XORing any bit with 1 produces its logical complement: A XOR 1 = NOT A. Consequently, XORing the state with 0FFh inverts every bit in the state. However, our macro is coded generically using two memory pointers (SI and DI) so that in Milestone 2 it seamlessly accepts any generated round key.

5. 8086 Architecture & Lecture Curriculum Mappings

Every instruction, addressing mode, and register used in this implementation adheres directly to the curriculum taught in CSIS 402 lectures:

Curriculum Concept	Lecture / Tutorial Source	Usage in AES Milestone 1
Data Segment Setup	Lecture 5 & Tutorial 7	MOV AX, @DATA / MOV DS, AX mandatory for DS initialization.
Register Indirect Addressing	Lecture 6 (Addressing Modes)	[BX] in SubBytes for accessing S-Box table elements.
Displacement / Relative	Lecture 6 (Addressing Modes)	[SI+disp] in ShiftRows and MixColumns to access column/row offsets.
Arithmetic & Logic	Tutorial 7 (ADD, SUB, XOR, etc.)	ADD BX, AX, XOR AL, AH, INC SI, DEC.
Bit Shifting & Flags	Lecture 5 & Lecture 8	SHL AL, 1 shifts MSB to Carry Flag (CF); JNC tests CF.
Iteration via LOOP	Lecture 8 (Jumps - Loops)	LOOP label decrements CX and branches if CX != 0.
Stack Operations	Lecture 5 (Registers & Stack)	PUSH CX / POP CX preserves outer loop counter during nested routines.
Subroutines vs. Macros	Lecture 8 & Project Requirements	Macros for SubBytes/ShiftRows/MixColumns/AddRoundKey; Procedure for xtime.

6. Compiling, Running, and Verification in MASM / DOSBox

To compile and execute the project using the **MASM/TASM extension** in VS Code:

1. Open aes_ms1.asm in VS Code.
2. Right-click anywhere in the editor and choose **Run ASM code** (or press Shift+F9 / extension shortcut).
3. To trace step-by-step, choose **Debug ASM code**. This launches the Turbo Debugger (TD) or DOSBox DEBUG session.

4. In the debugger, inspect the 16 bytes at memory label state before and after each macro call to verify correctness against known AES intermediate values.